SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED

CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 20%

QUESTIONS:5 Total Marks 20 Duration :60 Minutes Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I V.Thanusri(192472265) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Thanusri

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1.Real-Time Object Detection Pipeline
A system must:
A. Capture video frames
B. Detect objects using a trained model
C. Draw bounding boxes and labels
D. Display results in real time
Constraints:
- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

2. Image Difference Detection System
A system must:
A. Read two images
B. Compare them
C. Highlight differences
Constraints:
- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

3. Video Frame Extraction System
A system must:
A. Read video input
B. Extract frames at fixed intervals
C. Save selected frames
Constraints:
- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

4. Background Subtraction Pipeline
A surveillance system must:
A. Read video stream
B. Separate foreground from background
C. Highlight moving objects
Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

5. Edge Detection Pipeline for Real-Time Video

A system must:

A. Capture video frames

B. Apply edge detection

C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

Answers

1. Real-Time Object Detection Pipeline

Objective:

Capture live video, detect multiple objects using a trained model, draw bounding boxes and labels, and display the results efficiently in real time.

Modules:

1. Initialization and model loading
2. Video capture
3. Frame preprocessing
4. Object detection
5. Detection filtering
6. Visualization
7. Efficient looping and termination

Detailed Pseudocode:

```
BEGIN

    LOAD trained object detection model
```

```
LOAD class labels
SET confidence_threshold
SET NMS_threshold

OPEN video source

IF video source cannot be opened:
PRINT "Error: Cannot open video"
EXIT

WHILE video is available:

frame = READ next video frame

IF frame is empty:
BREAK

# Module 1: Efficient preprocessing
resized_frame = RESIZE(frame, model_input_size)
input_blob = PREPARE_INPUT(resized_frame)

# Module 2: Object detection
detections = MODEL_PREDICT(input_blob)

# Module 3: Filter detections
boxes = EMPTY_LIST
scores = EMPTY_LIST
class_ids = EMPTY_LIST

FOR each detection IN detections:

confidence = GET_CONFIDENCE(detection)

IF confidence >= confidence_threshold:

   class_id = GET_CLASS_ID(detection)
```

```
        box = CONVERT_TO_BOUNDING_BOX(detection, frame_size)
APPEND box TO boxes
        APPEND confidence TO scores
        APPEND class_id TO class_ids
# Module 4: Remove duplicate detections
selected_indices = APPLY_NON_MAXIMUM_SUPPRESSION(
                boxes, scores, NMS_threshold)
# Module 5: Visualization
FOR each index IN selected_indices:
box = boxes[index]
score = scores[index]
class_id = class_ids[index]
label = class_labels[class_id]
DRAW_RECTANGLE(frame, box)
DRAW_TEXT(frame, label + confidence_score, box)
# Module 6: Performance monitoring
CALCULATE FPS
DRAW FPS ON frame
# Module 7: Display
DISPLAY frame
key = READ_KEY()
IF key == EXIT_KEY:
BREAK
RELEASE video source
```

CLOSE all display windows

END

Efficiency and multiple-object handling:

• Resize frames to the model's required input size instead of processing unnecessarily large images.

• Use a lightweight trained model when CPU/GPU resources are limited.

• Apply confidence thresholding to reject weak detections.

• Use Non-Maximum Suppression (NMS) to remove overlapping duplicate boxes.

• Store only the current frame and current detections to limit memory usage.

• If required, process every second or third frame and use tracking between detections.

• Displaying FPS helps verify whether the system is meeting real-time requirements.

OpenCV mapping:

cv2.VideoCapture() → video input

cv2.resize() → resizing

cv2.dnn.blobFromImage() → model input preparation

net.forward() → model inference

cv2.dnn.NMSBoxes() → duplicate suppression

cv2.rectangle() → bounding boxes

cv2.putText() → labels

cv2.imshow() → real-time display

cv2.waitKey() → keyboard control

Conclusion:

The modular pipeline separates acquisition, detection, filtering, visualization, and looping. This makes the system easy to maintain while resizing, confidence filtering, NMS, and lightweight models keep processing efficient. Multiple objects are handled by storing and visualizing all valid detections in each frame.

2. Image Difference Detection System

Objective:

Read two images, preprocess them to reduce noise and illumination effects, compare corresponding pixels, highlight significant differences, and display the result.

Modules:

1. Image acquisition
2. Size and format normalization
3. Noise reduction
4. Image alignment
5. Difference calculation
6. Thresholding
7. Morphological filtering
8. Difference visualization

Detailed Pseudocode:

```
BEGIN

    image1 = READ_IMAGE("image1")
    image2 = READ_IMAGE("image2")

    IF image1 OR image2 cannot be read:
    PRINT "Error: Image loading failed"
    EXIT

    # Module 1: Normalize image size
    image2 = RESIZE(image2, SIZE_OF(image1))

    # Module 2: Convert to grayscale
    gray1 = CONVERT_TO_GRAYSCALE(image1)
```

```
gray2 = CONVERT_TO_GRAYSCALE(image2)

# Module 3: Reduce noise
gray1 = GAUSSIAN_BLUR(gray1, small_kernel)
gray2 = GAUSSIAN_BLUR(gray2, small_kernel)

# Module 4: Optional alignment
aligned_gray2 = ALIGN_IMAGE(gray2, gray1)

# Module 5: Calculate absolute difference
difference = ABSOLUTE_DIFFERENCE(gray1, aligned_gray2)

# Module 6: Threshold significant changes
binary_difference = THRESHOLD(
        difference,
          difference_threshold)

# Module 7: Remove small noise
binary_difference = MORPHOLOGICAL_OPENING(
          binary_difference)

# Fill small gaps in actual changed regions
binary_difference = MORPHOLOGICAL_CLOSING(
          binary_difference)

# Module 8: Find difference regions
contours = FIND_CONTOURS(binary_difference)

output = COPY(image2)

FOR each contour IN contours:

area = CONTOUR_AREA(contour)

IF area >= minimum_area:
```

```
        x, y, w, h = BOUNDING_RECTANGLE(contour)
        DRAW_RECTANGLE(output, x, y, w, h)
        OPTIONALLY:
            DRAW_TEXT(output, "Difference", x, y)
        # Module 9: Display
        DISPLAY image1
        DISPLAY image2
        DISPLAY binary_difference
        DISPLAY output
        WAIT FOR KEY
        CLOSE windows
END
```

Accuracy considerations:

- Both images should have the same dimensions before comparison.
- If the images come from different camera positions, image registration/alignment should be performed before subtraction.
- Gaussian or median filtering suppresses small sensor noise.
- Thresholding prevents tiny intensity variations from being interpreted as actual changes.
- Morphological opening removes isolated noise, while closing fills small holes.
- A minimum contour-area threshold prevents insignificant regions from being highlighted.
- For different lighting conditions, brightness normalization or histogram/CLAHE-based preprocessing can reduce false differences.

OpenCV mapping:

cv2.imread() → image reading

cv2.resize() → size normalization

cv2.cvtColor() → grayscale conversion

cv2.GaussianBlur() / cv2.medianBlur() → noise reduction

cv2.absdiff() → image difference

cv2.threshold() → binary difference

cv2.morphologyEx() → noise filtering

cv2.findContours() → difference regions

cv2.boundingRect() → region coordinates

cv2.rectangle() → highlight differences

Conclusion:

The system improves accuracy by aligning images, reducing noise, thresholding meaningful changes, and removing small regions using morphology and area filtering. The final image clearly highlights only significant differences.

3. Video Frame Extraction System

Objective:

Read a video, select frames at fixed time or frame intervals, save only the required frames, and avoid unnecessary storage and redundant frames.

Modules:

1. Video opening
2. Frame counting and FPS acquisition
3. Interval calculation
4. Frame selection
5. Duplicate/redundancy checking
6. Frame saving

7. Resource cleanup

Detailed Pseudocode:

```
BEGIN

    OPEN video = VideoCapture(video_path)

    IF video is not opened:
    PRINT "Error: Cannot open video"
    EXIT

    fps = GET_VIDEO_FPS(video)
    total_frames = GET_TOTAL_FRAME_COUNT(video)

    interval_seconds = USER_DEFINED_INTERVAL

    interval_frames = MAX(1, ROUND(fps * interval_seconds))

    frame_number = 0
    saved_count = 0
    last_saved_frame = NONE

    CREATE output_directory

    WHILE video has frames:

    frame = READ next frame

    IF frame is empty:
    BREAK

    # Select frames at fixed intervals
    IF frame_number MOD interval_frames == 0:

    # Optional redundancy test
    IF last_saved_frame IS NOT NONE:
```

```
            small_current = RESIZE(frame, small_size)
                small_previous = RESIZE(last_saved_frame, small_size)
            difference = MEAN_ABSOLUTE_DIFFERENCE(
                        small_current,
                        small_previous)
        IF difference < redundancy_threshold:
                frame_number = frame_number + 1
                CONTINUE
        filename = CREATE_FILENAME(saved_count, frame_number)
        SAVE frame TO output_directory/filename
        last_saved_frame = COPY(frame)
        saved_count = saved_count + 1
        frame_number = frame_number + 1
        RELEASE video
        PRINT "Total frames processed:", frame_number
        PRINT "Frames saved:", saved_count
END
```

Two frame-selection modes:

Mode 1 – Fixed frame interval:

Save every Nth frame.

Mode 2 – Fixed time interval:

interval_frames = FPS × desired_seconds

Save when current_frame >= next_save_frame

next_save_frame = next_save_frame + interval_frames

Storage optimization:

- Save only selected frames rather than every frame.
- Use JPEG for ordinary visual datasets where some compression is acceptable.
- Use PNG when lossless storage is important.
- Resize saved frames if full resolution is unnecessary.
- Use sequential filenames or timestamps for easy retrieval.
- Process one frame at a time so that the complete video is never stored in RAM.

Avoiding redundant frames:

A fixed interval alone does not guarantee that selected frames contain different content. Therefore, compare a small resized version of the current candidate frame with the previously saved frame. If the mean absolute difference is below a chosen threshold, the frame can be skipped.

OpenCV mapping:

cv2.VideoCapture() → video input

video.get() → FPS and frame count

cap.read() → frame extraction

cv2.resize() → compact redundancy comparison

cv2.imwrite() → frame saving

cap.release() → resource cleanup

Conclusion:

The algorithm provides predictable frame extraction using either frame-based or time-based intervals. Optional similarity checking further reduces redundant frames and storage requirements.

4. Background Subtraction Pipeline

Objective:

Read a surveillance video, estimate the background, identify moving foreground objects, reduce noise, and display highlighted moving regions while adapting to dynamic backgrounds.

Modules:

1. Video acquisition
2. Background model initialization
3. Foreground-mask generation
4. Noise filtering
5. Object extraction
6. Visualization
7. Continuous update

Detailed Pseudocode:

```
BEGIN

    OPEN video source

    IF video source cannot be opened:
    PRINT "Error"
    EXIT

    CREATE background_subtractor
    METHOD = MOG2 or KNN
    ENABLE_SHADOW_DETECTION = TRUE

    WHILE video is available:

    frame = READ next frame

    IF frame is empty:
    BREAK

    # Module 1: Preprocessing
```

```
    blurred_frame = GAUSSIAN_BLUR(frame, small_kernel)

    # Module 2: Background subtraction
    foreground_mask = background_subtractor.APPLY(
                blurred_frame,
                learning_rate)

    # Module 3: Remove shadows/noise
    foreground_mask = REMOVE_SHADOW_PIXELS(
                foreground_mask)
foreground_mask = MORPHOLOGICAL_OPENING(
                foreground_mask,
                small_kernel)
foreground_mask = MORPHOLOGICAL_CLOSING(
                foreground_mask,
                larger_kernel)

    # Optional dilation to connect fragmented objects
    foreground_mask = DILATE(foreground_mask)

    # Module 4: Detect foreground objects
    contours = FIND_CONTOURS(foreground_mask)

    output = COPY(frame)

    FOR each contour IN contours:

    area = CONTOUR_AREA(contour)

    IF area >= MIN_OBJECT_AREA:

    x, y, w, h = BOUNDING_RECTANGLE(contour)

      DRAW_RECTANGLE(output, x, y, w, h)
```

```
            DRAW_TEXT(output, "Moving Object", x, y)

    # Module 5: Display
    DISPLAY "Original", frame
    DISPLAY "Foreground Mask", foreground_mask
    DISPLAY "Moving Objects", output

    key = READ_KEY()

    IF key == EXIT_KEY:
    BREAK

    RELEASE video
    CLOSE windows
END
```

Handling dynamic backgrounds:

- MOG2 or KNN adapts the background model over time.
- A controlled learning rate allows gradual adaptation to moving trees, changing illumination, and other environmental changes.
- Shadow detection can reduce false foreground regions caused by shadows.
- Morphological operations eliminate small isolated pixels and reconnect fragmented objects.
- A minimum contour area prevents tiny background changes from being classified as objects.
- A region of interest can restrict processing to areas where movement is relevant.
- Temporal confirmation can be added so that an object must persist for several frames before triggering an alert.

Noise reduction:

Gaussian/median smoothing reduces sensor noise before subtraction. Opening removes

isolated foreground pixels, while closing fills small gaps. Area filtering eliminates very small contours.

OpenCV mapping:

cv2.VideoCapture() → video stream

cv2.createBackgroundSubtractorMOG2() → adaptive background model

cv2.GaussianBlur() → smoothing

cv2.morphologyEx() → filtering

cv2.dilate() → connect regions

cv2.findContours() → foreground objects

cv2.boundingRect() → object boxes

cv2.rectangle() → highlighting

cv2.imshow() → display

Conclusion:

Adaptive background subtraction combined with smoothing, shadow handling, morphology, and contour filtering provides a practical surveillance pipeline. MOG2/KNN continuously updates the background, allowing the system to handle moderate dynamic-background changes while maintaining useful real-time performance.

5. Edge Detection Pipeline for Real-Time Video

Objective:

Capture live video, preprocess each frame, detect clear edges, and display the result with sufficiently low processing time for real-time operation.

Modules:

1. Video acquisition
2. Frame resizing
3. Grayscale conversion

4. Noise reduction
5. Edge detection
6. Optional edge enhancement/cleanup
7. Visualization
8. Efficient loop

Detailed Pseudocode:

```
BEGIN

    OPEN camera or video stream

    IF camera cannot be opened:
    PRINT "Error: Cannot open video"
    EXIT

    SET lower_threshold
    SET upper_threshold

    WHILE video is available:

    frame = READ next frame

    IF frame is empty:
    BREAK

    # Module 1: Efficient resizing
    frame = RESIZE_IF_NEEDED(frame, working_resolution)

    # Module 2: Convert to grayscale
    gray = CONVERT_TO_GRAYSCALE(frame)

    # Module 3: Noise reduction
    gray = GAUSSIAN_BLUR(gray, small_kernel)

    # Module 4: Edge detection
```

```
        edges = CANNY(
                gray,
                lower_threshold,
                upper_threshold)

        # Module 5: Optional cleanup
        edges = MORPHOLOGICAL_CLOSE(edges, small_kernel)

        # Module 6: Visualization
        DISPLAY "Original Video", frame
        DISPLAY "Edges", edges

        # Optional overlay
        edge_overlay = CONVERT_EDGES_TO_COLOR(edges)
        combined = OVERLAY(frame, edge_overlay)

        DISPLAY "Edge Overlay", combined

        # Module 7: Real-time control
        key = READ_KEY()

        IF key == EXIT_KEY:
        BREAK

        RELEASE camera
        CLOSE all windows
END
```

Preprocessing justification:

- Grayscale reduces three color channels to one and therefore reduces computation.
- Gaussian blur suppresses small noise before Canny detection.
- The blur kernel should be small to preserve important boundaries.
- Resizing reduces the number of pixels processed per frame and improves FPS.

Edge detection:

Canny edge detection is suitable because it uses gradient information, non-maximum suppression, and hysteresis thresholding to produce relatively thin and clear edges. The lower and upper thresholds control sensitivity. If the thresholds are too low, noise edges appear; if too high, weak but important edges disappear.

Maintaining clarity:

- Use moderate Gaussian smoothing rather than excessive blur.
- Tune Canny thresholds according to scene brightness and noise.
- Optionally use automatic threshold selection based on the median image intensity.
- Morphological closing can connect small gaps in edges, but it should be used carefully because too much morphology can merge unrelated edges.
- Display the original and edge images simultaneously for visual verification.

Real-time optimization:

- Resize frames to a practical resolution.
- Use grayscale processing.
- Avoid expensive operations that do not contribute to edge quality.
- Process only a region of interest if edges are needed in a specific area.
- Use a small kernel for Gaussian filtering.
- Avoid accumulating frames in memory.
- Measure FPS and adjust resolution or processing frequency if performance falls.

OpenCV mapping:

cv2.VideoCapture() → video acquisition

cv2.resize() → frame resizing

cv2.cvtColor() → grayscale conversion

cv2.GaussianBlur() → noise reduction

cv2.Canny() → edge detection

cv2.morphologyEx() → optional edge cleanup

cv2.imshow() → visualization

cv2.waitKey() → loop control

cap.release() → resource release

Conclusion:

The grayscale → Gaussian blur → Canny → optional morphology pipeline provides clear edges with low computational cost. Frame resizing and simple operations help maintain real-time performance while appropriate threshold selection preserves important object boundaries.